

PENETRATION TESTING REPORT

Mediroza General Hospital
Web Application Security Assessment

Prepared by: Prathmesh Dilip Sargar
Cybersecurity Mentor: Waqas Karim, CCIE
Organisation: Networkwalks
Batch: B082 | Week 4 Capstone Project
Target: <https://medirozahospital.com>
Classification: Confidential

1. Executive Summary

I was assigned to conduct a black-box penetration test on the web infrastructure of Mediroza General Hospital at <https://medirozahospital.com>. The client provided written authorisation for this assessment. The goal was to identify vulnerabilities, demonstrate their real-world impact through exploitation, and provide recommendations to improve the security posture of the organisation.

During the assessment I identified seven vulnerabilities ranging from Medium to Critical severity. The most significant finding was a SQL injection vulnerability on the patient portal login page, which allowed me to bypass authentication entirely without knowing any credentials. This gave me access to three confidential patient lab report PDFs. After cracking the PDF encryption using password cracking tools, I discovered sensitive metadata inside one of the files that pointed to a forgotten database backup stored in a publicly accessible folder on the server. This backup contained the monthly salaries of all 30 hospital employees and the shareholding details of 10 shareholders.

The overall security posture of the target is poor. Multiple critical vulnerabilities exist that would allow an unauthenticated attacker to access confidential patient data, staff financial records, and corporate ownership information with minimal effort and no specialised equipment.

Overall Risk Rating: CRITICAL — Immediate remediation is recommended.

2. Scope and Methodology

2.1 Scope

The assessment was limited to the following target as agreed with the client.

Target domain: <https://medirozahospital.com>

The following were excluded from scope: social engineering, denial of service attacks, and any testing outside the agreed domain.

2.2 Methodology

I followed a structured black-box penetration testing methodology consisting of four phases.

- **Reconnaissance:** Passive information gathering using publicly available information and web-based tools.
- **Vulnerability Identification:** Analysing the application behaviour to find weaknesses in authentication and input handling.
- **Exploitation:** Demonstrating the real impact of each vulnerability through controlled exploitation.
- **Documentation:** Recording all findings, evidence, and remediation recommendations in this report.

2.3 Tools Used

- curl — command line tool for sending HTTP requests and reading web server responses
- Browser developer tools — for inspecting page source and login form behaviour
- Networkwalks Hash Calculator — for extracting password hashes from PDF files
- Networkwalks Password Cracker — for cracking PDF password hashes using wordlists
- qpdf — for decrypting password-protected PDF files after cracking
- exiftool — for reading hidden metadata from PDF files
- wget — for downloading files from the web server
- ChatGPT — for converting raw SQL data into readable tables

3. Findings and Proof of Exploitation

3.1 Summary Table

#	Vulnerability	Location	Risk
1	Username enumeration on login page	patient/login.php	Medium
2	SQL injection login bypass	patient/login.php	Critical
3	Encrypted PDFs accessible after login bypass	patient/reports/	High
4	Weak PDF passwords crackable with a wordlist	patient_report_*.pdf	High
5	Sensitive metadata left in patient PDF files	patient_report_3.pdf	Medium
6	Forgotten backup folder with directory listing enabled	old/	Critical
7	Confidential staff salaries and shareholder data in plain text	plaid/mediroza_db_backup_2019.sql	Critical

3.2 Finding 1 — Username Enumeration

Risk Rating: Medium

Location: patient/login.php

Description

Username enumeration occurs when a login page reveals whether a username exists or not by showing different error messages for a wrong username versus a wrong password. A secure login should always show the same generic message for both cases to prevent an attacker from confirming valid usernames.

Steps Taken

I opened the patient portal login page and tested the error messages by entering different inputs.

First I entered a username that I assumed would not exist.

```
username: bob
password: test123
```

RESPONSE

Username not found

Then I tried a common default username.

```
username: admin
password: test123
```

RESPONSE

Incorrect password

The two different messages confirmed that admin is a valid account on the system. I now had the username and only needed to find the password.

[Screenshot: Browser showing "Username not found" for bob]

[Screenshot: Browser showing "Incorrect password" for admin]

3.3 Finding 2 — SQL Injection Login Bypass

Risk Rating: Critical

Location: patient/login.php

Description

SQL injection is a vulnerability where an application places user input directly inside a database query without sanitising it first. An attacker can insert SQL code into the input field to change the behaviour of the query. In this case, I was able to comment out the password check entirely and log in as admin without knowing the password.

Steps Taken

I tested the username field for SQL injection by typing a single quote.

```
username: admin'  
password: test123
```

RESPONSE

```
Warning: mysqli_query(): You have an error in your SQL syntax; check the manual  
that corresponds to your MySQL server version for the right syntax to use near  
''' at line 1
```

The database error confirmed the field was injectable. The application was building its SQL query like this behind the scenes.

```
SELECT * FROM users WHERE username='admin'' AND password='test123'
```

The extra quote broke the query and caused the error. I then crafted the classic SQL injection bypass payload.

```
username: admin' --  
password: anything
```

The -- comments out everything after it in SQL, so the query becomes.

```
SELECT * FROM users WHERE username='admin'
```

The password check is ignored completely. The query returns the admin row and I am logged in.

[Screenshot: Database error when single quote is entered]

[Screenshot: Logged in portal page after SQL injection]

3.4 Finding 3 — Confidential PDFs Accessible After Login Bypass

Risk Rating: High

Location: patient/reports/

Description

After gaining unauthorised access to the portal through SQL injection, I found three patient lab report PDFs available for download. These are confidential medical documents that should only be accessible to the named patients and their doctors.

Steps Taken

After logging in with the SQL injection payload I was directed to the patient portal which listed three downloadable PDF files.

```
patient_report_1.pdf  
patient_report_2.pdf  
patient_report_3.pdf
```

I downloaded all three files.

[Screenshot: Portal page showing the 3 downloadable reports]

[Screenshot: Downloaded files in the Downloads folder]

3.5 Finding 4 — Weak PDF Passwords Crackable with a Wordlist

Risk Rating: High

Description

All three PDFs were password protected. However the passwords were weak and appeared in commonly available password wordlists, making them trivial to crack using automated tools. Using simple passwords on sensitive medical documents does not provide meaningful protection.

Steps Taken

I used the Networkwalks Hash Calculator to extract a crackable hash from each PDF, then ran each hash through the Networkwalks Password Cracker.

Reports 1 and 2 cracked immediately using the built-in 100 word default wordlist.

RESULTS

```
patient_report_1.pdf → 123456  
patient_report_2.pdf → password
```

Report 3 did not crack with the built-in list. I switched to a larger wordlist (JTR default password list) and ran the attack again.

```
patient_report_3.pdf → !@#$$%^&
```

I then opened each PDF with its cracked password and read the confidential patient medical data inside.

[Screenshot: Networkwalks Password Cracker showing cracked password for report 1]

[Screenshot: Networkwalks Password Cracker showing cracked password for report 3 with JTR list]

[Screenshot: Opened PDF showing patient medical data]

3.6 Finding 5 — Sensitive Metadata in Patient PDF Files

Risk Rating: Medium

Description

PDF files store hidden metadata such as the author name, creation date, and custom comments. This metadata is not visible when simply reading the document but can be extracted using tools like exiftool. In this case, report 3 contained an internal staff comment left by the IT administrator that pointed directly to a confidential backup file on the server.

Steps Taken

I first decrypted report 3 using qpdf so that exiftool could read all metadata fields. A locked PDF only shows the encryption type.

```
qpdf --password='!@#$$%^&' --decrypt patient_report_3.pdf report3_open.pdf
```

Then I ran exiftool on the unlocked file.

```
exiftool report3_open.pdf
```

KEY FIELDS IN THE OUTPUT

Author : j.malik

Comments : DB backup moved to /old before site migration, do not delete

This comment was left by the IT administrator Jameel Malik (j.malik) and revealed the exact location of a database backup on the server.

[Screenshot: exiftool output showing Author j.malik and Comments field]

3.7 Finding 6 — Forgotten Backup Folder with Directory Listing Enabled

Risk Rating: Critical

Description

Directory listing is a web server misconfiguration that displays the contents of a folder like a file browser when no index page exists. The /old folder on the server had directory listing enabled, exposing a database backup file to anyone who visited that URL. I had already found this folder during my initial recon from the robots.txt file, which listed it as a disallowed path.

Steps Taken

During recon I noted /old in the robots file. The metadata clue from Finding 5 confirmed this was the location of the backup. I opened the folder in the browser.

```
https://medirozahospital.com/old/
```

The directory listing was enabled and I could see the backup file immediately.

```
mediroza_db_backup_2019.sql
```

I downloaded it using wget.

```
wget https://medirozahospital.com/old/mediroza_db_backup_2019.sql
```

[Screenshot: Browser showing /old/ directory listing with the SQL file visible]

3.8 Finding 7 — Confidential Staff Salaries and Shareholder Data in Plain Text

Risk Rating: Critical

Description

The database backup contained highly sensitive information in plain text including the full name, job title, email address, phone number, national ID number, and monthly salary in ZAR for all 30 hospital employees. It also contained the names and shareholding percentages of all 10 shareholders of the hospital. This data was completely unprotected and accessible to anyone who visited the /old folder.

Steps Taken

I opened the SQL file in a text editor. The data was in raw SQL format. I copied the staff INSERT INTO rows and used ChatGPT to convert them into a readable table.

```
Present this SQL data as a readable table showing name, job title, department and monthly salary.
```

Then I did the same for the shareholders.

```
Present this SQL data as a readable table showing shareholder name, share percentage and share class.
```

I was able to read the private salary of every employee and the ownership structure of the entire hospital.

I also confirmed that the Author field in the PDF metadata, j.malik, corresponds to Jameel Malik, the IT Systems Administrator listed in the staff table. This connects all three milestones in a single attack chain.

[Screenshot: ChatGPT output showing staff salary table]

[Screenshot: ChatGPT output showing shareholder table]

4. Full Attack Chain Summary

The following is the complete path from zero access to full data exposure, showing how each finding led to the next.

- **Step 1:** Robots.txt revealed three hidden folders: /patient, /staff, and /old.
- **Step 2:** The patient login page showed different error messages for wrong username and wrong password, confirming admin as a valid account. (Finding 1)
- **Step 3:** A single quote in the username field triggered a database error, confirming SQL injection. (Finding 2)
- **Step 4:** The payload admin' -- bypassed the login entirely and gave access to the patient portal. (Finding 2)
- **Step 5:** Three confidential patient PDF lab reports were downloaded from the portal. (Finding 3)
- **Step 6:** The Networkwalks Password Cracker cracked the PDF passwords using wordlists. Reports 1 and 2 used the built-in list. Report 3 required the larger JTR list. (Finding 4)
- **Step 7:** exiftool on the unlocked report 3 revealed a hidden comment by j.malik pointing to /old. (Finding 5)
- **Step 8:** The /old folder had directory listing enabled, exposing the database backup. (Finding 6)
- **Step 9:** The backup contained salaries of 30 employees and shareholding details of 10 shareholders in plain text. (Finding 7)

5. Recommendations and Remediation

5.1 Fix Username Enumeration

Change the login page to show a single generic message for any failed login attempt, regardless of whether the username or password was wrong. For example: "Invalid credentials. Please try again."

5.2 Fix SQL Injection

Replace the current login query with a parameterised query or prepared statement. This separates the SQL code from the user input so that no amount of crafted input can change the query structure. This is the most important fix in this report.

```
// Safe example using PHP PDO prepared statement
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?")
$stmt->execute([$username, $password])
```

5.3 Fix PDF Access and Password Strength

Move the PDF files outside the web root so they cannot be served directly by the web server. Enforce access through a server-side script that checks authentication before serving any file. If passwords are used, enforce a minimum length of 12 characters with a mix of uppercase, lowercase, numbers, and symbols.

5.4 Strip PDF Metadata

Remove all metadata from files before distributing them. Staff notes or internal comments must never be stored inside files that leave the organisation. Use the following command to strip all metadata.

```
exiftool -all= patient_report_3.pdf
```

5.5 Fix Directory Listing and Remove the Backup

Disable directory listing on all folders by adding Options -Indexes to the Apache configuration or .htaccess file. Delete the database backup from the /old folder immediately. Database backups must never be stored inside the web root. Store them in a private, access-controlled location outside the publicly accessible part of the server.

6. Conclusion

This assessment found a complete attack chain from the login page to highly sensitive internal data. No sophisticated tools or knowledge were required. All vulnerabilities found in this report are well-known and have well-established fixes. I recommend the client address all Critical and High findings immediately before the system is used to store or serve real patient data.

Submitted by: Prathmesh Dilip Sargar

Cybersecurity Mentor: Waqas Karim, CCIE

Organisation: Networkwalks

Batch B082 | Week 4 Capstone Project

This report is produced as part of a controlled educational exercise by Networkwalks. The target has been authorised for security testing. These techniques must never be applied to any system without explicit written permission from the owner.

This report is submitted as part of the Networkwalks B082 Cybersecurity Internship Week 4 Capstone Project. All testing was conducted in a controlled environment with written authorisation from the client. These techniques must never be applied to any system without explicit written permission from the owner.